Résumé

Les concepts clés du développement Jakarta EE — focus Servlets

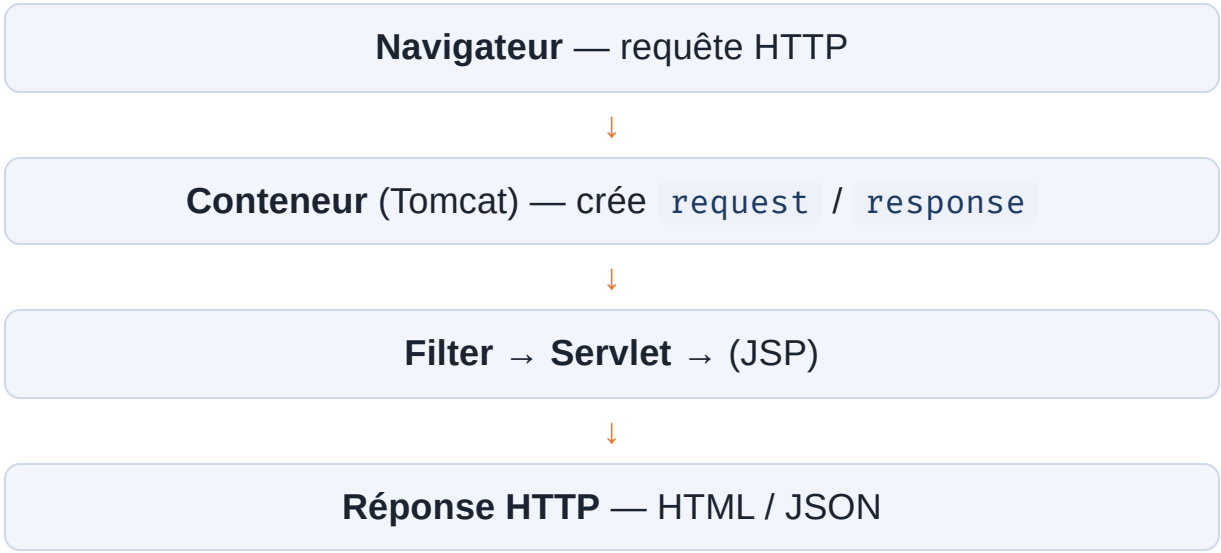
Container · Servlet · Request / Response · Session · Filter · MVC

Computer Science — Jakarta EE | Lecture 02
Fiche de révision pour les étudiants

Le tableau d'ensemble

- **Jakarta EE** = des **spécifications** (pas un produit).
- Un **conteneur web** (Tomcat 11) implémente **Servlet 6.1** et **JSP**.
- Votre code = des **classes Java** déployées dans un **WAR**.
- Le conteneur gère : **réseau, threads, cycle de vie, sessions, sécurité**.

> Vous écrivez la **logique** ; le conteneur appelle **votre** code (inversion de contrôle).



Rappel HTTP : ce que reçoit une servlet

```
POST /calculatrice/calc HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Cookie: JSESSIONID=ABC123

a=7&b=2&op=mul
```

Partie HTTP	API Servlet
Méthode	`req.getMethod()`
Chemin	`req.getRequestURI()`
Paramètres	`req.getParameter("a")`
En-têtes	`req.getHeader("Cookie")`
Corps	`req.getReader()`
Cookies	`req.getCookies()`

Réponse : code (`200` , `302` , `404` …), en-têtes, corps.

Anatomie d'une Servlet

```java
@WebServlet(name = "calculatrice", urlPatterns = "/calc")      // 1. mapping URL
public class CalculatriceServlet extends HttpServlet {          // 2. hérite de HttpServlet

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws IOException {                                 // 3. une méthode par verbe HTTP
        resp.setContentType("text/html;charset=UTF-8");          // 4. type + encodage
        resp.getWriter().print("<h1>Calculatrice</h1>");         // 5. écrire le corps
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) { /* ... */ }
}
```

Verbe	Méthode	Usage typique
GET / POST	`doGet` / `doPost`	afficher / soumettre un formulaire
PUT / DELETE	`doPut` / `doDelete`	modifier / supprimer (API REST)

Cycle de vie & concurrence

Chargement — 1 instance

↓

init() — une fois

↓

service() → `doGet` / `doPost`
à chaque requête, dans un thread du pool

↓

destroy() — à l'arrêt

- **Une seule instance** par servlet, **plusieurs threads** en parallèle.
- ❌ Pas d'état modifiable dans des **attributs d'instance**.
- ✅ État par requête → **variables locales**.
- ✅ État par utilisateur → **`HttpSession`**.

```
// DANGER : partagé entre tous les utilisateurs !
private int compteur = 0;
```

Une **race condition** sur un champ de servlet est l'erreur n°1 des débutants.

Request, Response & les 4 portées

```
String a  = req.getParameter("a");   // entrée
req.setAttribute("res", 15);          // partage interne
resp.setStatus(200);
resp.sendRedirect("calc");            // 302
```

- **Toujours** valider les paramètres : `getParameter` renvoie `null` ou du texte.
- **Toujours** échapper la sortie HTML (anti-**XSS**).
- `setCharacterEncoding("UTF-8")` avant de lire le corps.

Portée	Objet	Durée de vie
Page	`PageContext`	la page JSP
Request	`HttpServletRequest`	une requête
Session	`HttpSession`	un utilisateur
Application	`ServletContext`	toute l'appli

`setAttribute` / `getAttribute` existent sur chacun d'eux.

Sessions & état utilisateur

HTTP est **sans état**. Le conteneur ajoute l'état avec un cookie `JSESSIONID`.

```
HttpSession session = req.getSession();       // crée si absente
List<String> h = (List<String>)
        session.getAttribute("history");
session.setAttribute("history", h);
session.invalidate();                          // déconnexion
```

1 1re requête → pas de cookie

↓

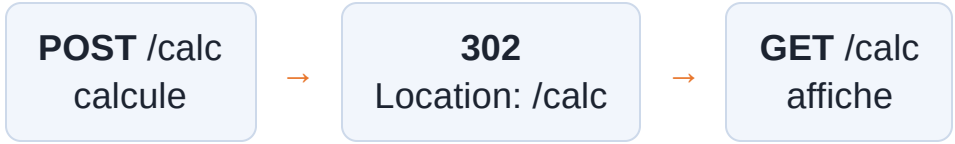
2 `Set-Cookie: JSESSIONID=ABC`

↓

3 requêtes suivantes envoient le cookie

Exemple : l'historique de la **calculatrice** vit dans la session.

Le pattern Post / Redirect / Get



- Évite le **double envoi** quand l'utilisateur actualise la page.
- Le résultat / l'erreur passe par la **session** (message « flash »), puis est supprimé.

```
session.setAttribute("error", "Division par zéro impossible");
resp.sendRedirect(req.getContextPath() + "/calc");   // jamais de logique après un redirect
```

Forward, Redirect & MVC

	`forward`	`sendRedirect`
Où ?	**serveur**	**navigateur**
URL change	non	oui
Requêtes	1	2
Partage `request`	oui	non

```
req.setAttribute("res", res);
req.getRequestDispatcher("/WEB-INF/result.jsp")
   .forward(req, resp);
```

MVC avec Jakarta EE

Model — classes Java (POJO)

↑↓

Controller — Servlet

↓

View — JSP (dans `WEB-INF`)

Servlet = logique, JSP = affichage. Ne pas mélanger.

Filters, Listeners & configuration

Filter — code avant / après la servlet (log, encodage, auth) :

```
@WebFilter("/*")
public class LogFilter implements Filter {
  public void doFilter(ServletRequest rq,
        ServletResponse rs, FilterChain chain)
        throws IOException, ServletException {
    // avant
    chain.doFilter(rq, rs);
    // après
  }
}
```

Listener — réagit aux événements (`ServletContextListener`, `HttpSessionListener`).

Configuration

Moyen	Exemple
Annotation	`@WebServlet("/calc")`
`web.xml`	`<welcome-file-list>`, erreurs
Init param	`@WebInitParam`

> Les annotations suffisent depuis Servlet 3.0 ; `web.xml` sert aux réglages globaux.

Projet : Calculatrice (Servlets + Tailwind)

```
calculatrice/
├── pom.xml
├── package.json          (Tailwind CLI)
└── src/main/
    ├── java/.../calculatrice/
    │   ├── CalculatriceServlet.java
    │   ├── HistoryServlet.java
    │   ├── Operation.java
    │   └── Layout.java
    ├── tailwind/input.css
    └── webapp/
        ├── css/styles.css   (généré)
        └── WEB-INF/web.xml
```

- `GET /calc` → formulaire + historique.
- `POST /calc` → calcul, historique en **session**, **redirect**.
- `POST /history` → efface l'historique.
- **Tailwind** : `npm run css` scanne les classes écrites dans les servlets.

```
cd calculatrice
npm install && npm run css
mvn package      # target/calculatrice.war
```

Bonnes pratiques & erreurs fréquentes

✅ À faire

- Valider **toutes** les entrées (`null`, format, division par 0).
- Échapper la sortie HTML (**XSS**).
- `setContentType` + **UTF-8** partout.
- Post/Redirect/Get après un `POST`.
- Séparer logique (Java) et présentation.
- Garder les servlets **minces** : déléguer à des classes métier.

❌ À éviter

- État modifiable dans les champs de la servlet.
- Écrire tout le HTML dans la servlet (OK pour apprendre, pas en production → JSP).
- Oublier `return` après `sendRedirect` / `sendError`.
- Avaler les exceptions sans message utile.
- Faire confiance aux paramètres du client.

À retenir

1. Le **conteneur** appelle vos servlets ; vous ne les instanciez jamais.
2. **Une instance, plusieurs threads** → pas d'état partagé.
3. `doGet` / `doPost` lisent la **request**, écrivent la **response**.
4. **Session** = état par utilisateur ; **Application** = état global.
5. **Forward** (interne) vs **Redirect** (navigateur).
6. **Filter** pour le code transversal ; **MVC** pour l'organisation.

Références

- Servlet 6.1 — https://jakarta.ee/specifications/servlet/6.1/
- Jakarta EE Tutorial — https://eclipse-ee4j.github.io/jakartaee-tutorial/
- Apache Tomcat 11 — https://tomcat.apache.org/
- Tailwind CSS — https://tailwindcss.com/

Exercices

- Ajouter l'opération **modulo** et **puissance** à `Operation`.
- Ajouter un `LogFilter` qui mesure le temps de chaque requête.
- Déplacer le HTML de la calculatrice dans une **JSP** (forward).

Code : `hello-world/` et `calculatrice/` dans ce dépôt.